PaaS4Bat PINN Case Study: Battery Capacity Fade Prediction

Osten Jap

27.07.2026-02.08.2026

Abstract

We study lithium-ion battery capacity fade using cycle data from the NASA PCoE cycling dataset, with a contiguous train/test split (cycles 2–90 train, 91–132 held out) chosen so that evaluation measures genuine extrapolation rather than interpolation. As baselines, we fit a plain feedforward network mapping cycle number directly to capacity and a windowed LSTM that forecasts capacity recursively from recent readings; both reach a held-out RMSE of roughly 0.03–0.07 Ah despite having no notion of the underlying degradation physics. Against these we develop a physics-informed neural network whose latent state is the growing solid-electrolyte-interphase (SEI) film, governed by a diffusion-limited growth law that we integrate in $u = \delta^2$ to remove the $1/\delta$ singularity and from which capacity is derived rather than predicted directly. A hard initial-condition ansatz fixes $\mathrm{SOH}(0) = 1$ exactly, and the two physical scalars (k_{ref}, c_{Li}) are learned jointly with the network under a physics-residual penalty whose weight is chosen for parameter identifiability rather than fit alone. The result is a model that extrapolates under an interpretable kinetic constraint. That weight was set on synthetic data with a known ground-truth rate, where agreement between independently initialised runs distinguishes a physics term that is genuinely enforced from one that is decorative; on the measured cell we instead verify the constraint directly, by checking the invariants the model is supposed to respect.

1 Introduction & Literature Review

1.1 Problem statement

Accurate capacity-fade prediction underpins safe, cost-effective deployment of Li-ion batteries across EVs, grid storage, and consumer electronics: it drives state-of-health estimation and remaining-useful-life forecasts that determine warranty terms, second-life viability, and replacement schedules. Getting these predictions wrong either strands usable capacity through premature retirement or risks safety incidents from batteries pushed past their true end of life. In this work we use machine learning models – a feedforward network and an LSTM – to predict capacity fade from cycling data, and we evaluate whether their extrapolation to unseen, later cycles is reliable enough to trust. We then examine whether embedding known degradation physics into the model, rather than relying on pattern-matching alone, improves that extrapolation and yields predictions that respect physically plausible behaviour.

1.2 PINN architectures reviewed

- **Wang et al. (2024)** [1] – physics constraint embedded: a degradation-consistency residual enforcing monotone, physically stable capacity fade within the network's training loss. *[Adopted / not adopted because ...]*

1.3 Chosen architecture and governing equation

Rather than design a new network from scratch, we start from the plain MLP of Section 2.3 – already validated as a working curve-fitter on this exact data and split – and add physics on top of it, changing what the output means and what the loss penalises. The network is not held fixed, however: the physics term requires smooth activations throughout, and the model that follows is one hidden layer deeper with a `sigmoid` final hidden layer (Section 2.2). The PINN-vs-MLP comparison in Section 3 is therefore *not* a single-variable ablation, and we do not attribute the difference to the governing equation alone; isolating the physics term would require a PINN built on the baseline's exact architecture, which we flag as follow-up work rather than claim here.

We model diffusion-limited SEI growth (see `docs/decisions.md`, D-002), rejecting a lithium-plating term because the NASA aging protocol holds charge current and ambient temperature fixed within each group, leaving no experimental contrast from which to identify a plating exchange-current density (D-002). In its simplest, constant-rate form the growth law is written per cycle n rather than per unit wall-clock time, since the implemented model (`documents/step4_sei_growth_pinn.ipynb`) has not yet brought in a measured temperature signal to support an Arrhenius term (that extension is left to `docs/step9` "future work - out of scope"). Solving in $u = \delta^2$ removes the $1/\delta$ singularity in the textbook growth law (D-009):

$$\frac{du}{dn} = 2k_{\text{ref}}, \qquad \delta = \sqrt{u}, \qquad Q(n) = Q_0 - c_{\text{Li}}\,(\delta(n) - \delta_0). \tag{1}$$

k_{ref} (the SEI growth rate) and c_{Li} (Ah of capacity lost per unit film growth) are the two physical unknowns identified from data; both are stored as $\log k_{\text{ref}}$, $\log c_{\text{Li}}$ during training so that exponentiating on read-out makes a negative rate or coefficient unrepresentable rather than merely discouraged. Extending Eq. 1 with an Arrhenius temperature dependence and a literature activation energy E_a is future work once a multi-temperature or EIS-derived signal is available (D-006).

2 Methodology

2.1 Dataset

NASA PCoE cell B0005 cycling data, extracted by `src/fetch_nasa.py`/`src/explore_nasa.py` (capacity asserted within 0.5 A h to 3.0 A h) and validated by `documents/dataExtractionAndCleaning.ipynb` (monotonic `cycle` and `elapsed_h`, SOH $\in (0, 1.05]$, non-negative `rest_h`, `delta_capacity` recomputed from the capacity series rather than from SOH), yielding `data/processed/clean_data.csv`: 131 discharge cycles (2–132), capacity/SOH/rest-hours/elapsed-time per row. No cycle-1 row exists – the earliest is cycle 2 (1.8432 A h, SOH = 0.9936) – so the SOH = 1 reference anchoring Section 2.2's hard initial condition is Q_0 = capacity_ah/SOH = 1.855 A h, constant across all 131 rows. Split contiguous by cycle: 2–90 train (89 pts), 91–132 held out (42 pts), measuring extrapolation rather than interpolation. Cycle number is normalised $\tau = n/n_{\max}$ on $n_{\max} = 90$ (training range only), so held-out cycles reach $\tau \approx 1.47$ – the overshoot past 1.0 being tested.

2.2 PINN architecture

The network (`documents/step4_sei_growth_pinn.ipynb`) maps normalised cycle $\tau \in [0, 1]$ to the SEI film state $u = \delta^2$ through four hidden layers of 32 units each – three `tanh` followed by one `sigmoid` – one layer deeper than the plain baseline network of Section 2.3. Both activations are smooth everywhere, which matters because the physics loss backpropagates through $du/d\tau$ (via `autograd.grad` with `create_graph=True`), pulling in the activations' second derivatives; `ReLU`'s

second derivative is zero almost everywhere and undefined at its kinks, so it cannot support this term. The network never outputs capacity directly. A hard initial-condition ansatz,

$$u(\tau) = u_0 + \tau \cdot \mathrm{softplus}\big(\mathrm{net}(\tau)\big), \tag{2}$$

fixes $\mathrm{SOH}(0) = 1$ exactly for every possible network weight (the added term vanishes identically at $\tau = 0$) rather than as something the optimiser has to learn, and `softplus` guarantees the film can only thicken. Capacity is then derived, $Q(\tau) = Q_0 - c_{\mathrm{Li}}\,(\delta(\tau) - \delta_0)$, with $Q_0 = 1.855\,\mathrm{A\,h}$ fixed from the SOH = 1 reference of Section 2.1. The two physical scalars $\log k_{\mathrm{ref}}$, $\log c_{\mathrm{Li}}$ are learned jointly with the network (exponentiated on read-out to stay strictly positive). The physics residual is evaluated on 256 unlabelled collocation points spanning $\tau \in [0, 1]$,

$$\mathcal{L} = \mathcal{L}_{\mathrm{data}} + \lambda_{\mathrm{phys}}\,\mathcal{L}_{\mathrm{phys}}, \qquad \mathcal{L}_{\mathrm{phys}} = \mathrm{mean}\left[\left(\frac{du}{d\tau}\Big/ n_{\max} - 2k_{\mathrm{ref}}\right)^2\right], \tag{3}$$

where the $n_{\max}$ rescaling converts the raw τ-derivative into a true per-cycle rate. $\lambda_{\mathrm{phys}} = 100$ is carried over as-is from the synthetic parameter-recovery sweep in `src/warmup_pinn.py` (D-012), not re-tuned on this dataset. Training uses Adam, $\mathrm{lr} = 5 \times 10^{-3}$, 3000 epochs, seed 0 – identical optimiser, epoch count, and seed to the plain MLP baseline, so that any accuracy difference traces to the change in loss and architecture, not training budget.

2.3 Baseline models

- **MLP** (`documents/step1_baseline_network.ipynb`) – 3 hidden layers of 32 `tanh` units mapping normalised cycle $\tau = n/n_{\max}$ directly to capacity, trained on plain MSE (Adam, $\mathrm{lr} = 5 \times 10^{-3}$, 3000 epochs, seed 0). The PINN of Section 2.2 shares its optimiser, epoch count and seed but not its shape – it is one layer deeper and ends in a `sigmoid` – so the two differ in architecture as well as loss. A `ReLU` variant of the same shape is also trained as a secondary control, since `ReLU`'s sharper bends can fit this non-smooth curve better in the absence of a physics term that needs a second derivative.
- **LSTM** (`documents/step2_lstm_baseline.ipynb`) – a 2-layer `nn.LSTM` with hidden size 32 followed by a linear read-out of the final hidden state, trained on sliding windows of length 5 built from the 89 training points (84 overlapping windows), again with plain MSE (Adam, $\mathrm{lr} = 5 \times 10^{-3}$, 1500 epochs, seed 0). Unlike the MLP, it has no way to evaluate an arbitrary cycle directly, so the 42 held-out cycles are forecast recursively – each prediction is fed back in as if it were observed data to produce the next one – meaning early errors in the held-out region compound into everything after them.

3 Results

3.1 Error metrics

Table 1: Held-out prediction error, PINN vs. baselines. Every row is the value reported by its own source notebook.

Model	RMSE (Ah)	MAE (Ah)	MAPE (%)
MLP (tanh)	0.0711	0.0602	4.32
MLP (ReLU)	0.0353	0.0260	1.85
LSTM	0.0363	0.0291	2.08
PINN (A1, soft constraint)	0.0457	0.0387	2.78

LSTM trained on GPU. On CPU the same code gives 0.0336 / 0.0257 / 1.82 – cuDNN just doesn't reproduce CPU bit-for-bit, so treat LSTM and MLP (ReLU) as tied rather than ranked.

3.2 Capacity fade curves

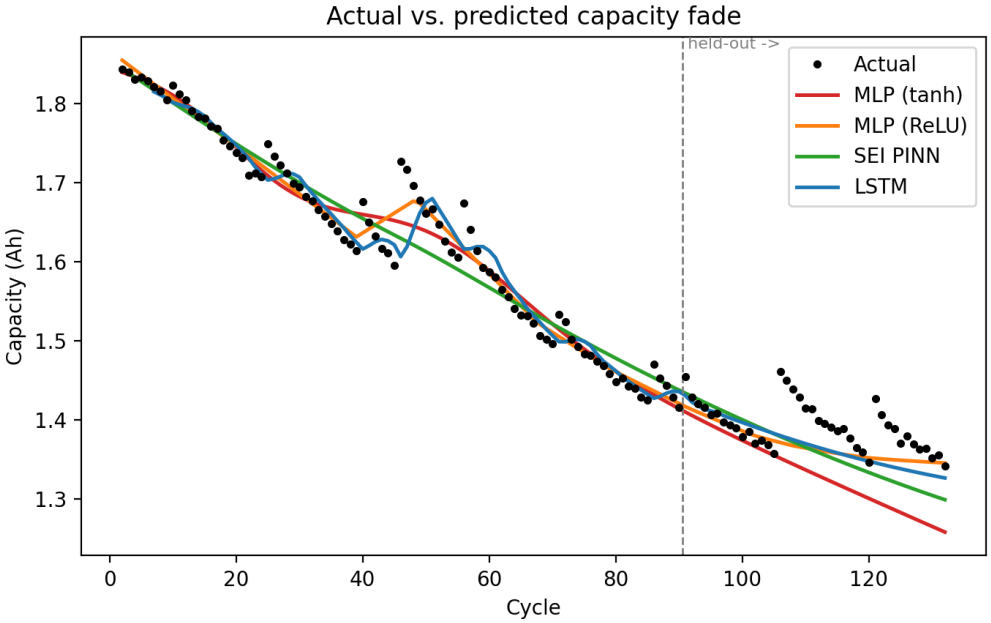


Figure 1: Actual vs. predicted capacity fade for all four models (MLP tanh, MLP ReLU, LSTM, SEI PINN), regenerated with the source notebooks' own definitions and on the same devices, so each curve reproduces its Table 1 row exactly (the LSTM on GPU, at 0.0363 / 0.0291 A h). MLP and PINN curves evaluate the full cycle range directly; the LSTM curve is teacher-forced (one-step-ahead on true history) over the training windows and switches to the same recursive forecast used for its Table 1 metrics once past cycle 90 – it has no curve for cycles 2–6 since those are only ever consumed as window input, never predicted.

4 Discussion

4.1 Physical consistency check

1. Does the PINN respect physical limits? Yes, though only partly by construction. Two things are structural: the hard ansatz $u = u_0 + \tau\,\mathrm{softplus}(\mathrm{net}(\tau))$ pins $\mathrm{SOH}(0) = 1$ exactly and keeps $u \geq u_0$, so capacity can never exceed Q_0; and $c_{\mathrm{Li}} > 0$ via the log parameterisation, so the sign of the fade term cannot flip. Monotonicity at interior cycles is *not* guaranteed – differentiating the ansatz gives $du/d\tau = \mathrm{softplus}(\mathrm{net}) + \tau\,\sigma(\mathrm{net})\,\mathrm{net}'(\tau)$, whose second term may be negative – so it is enforced only softly, by the residual $du/dn = 2k_{\mathrm{ref}}$, and must be verified rather than assumed. It holds here: measured over cycles 2–132 the PINN curve has **0** capacity-increase steps, whereas MLP (ReLU) – the best baseline on RMSE – has **9**, i.e. it predicts a battery that spontaneously heals. Removing the interior-monotonicity caveat entirely is the motivation for the hard-monotone variant (A2).

2. Does it behave reasonably under extrapolation? Across the evaluated range, yes. At the last held-out cycle (132, $\tau \approx 1.47$) the PINN predicts 1.299 A h against an actual 1.341 A h, while the plain MLP (tanh) baseline undershoots further to 1.258 A h, and the PINN commits **0** monotonicity violations over cycles 2–132. Pushed far beyond any data, to cycle 1000, it stays bounded and strictly positive (0.47 A h) rather than diverging – the $\sqrt{u}$ form makes fade decelerate as the film grows, as diffusion-limited growth should – but **32** small non-monotone steps appear. That is precisely the caveat above becoming visible: the soft residual is enough

to keep the film well-behaved where it is anchored by data, and not enough 7× past it. For that far-extrapolation regime specifically, monotonicity therefore has to be added explicitly – built into the functional form as in the hard-monotone A2 variant – rather than left to the physics penalty to enforce.

3. How does the physics constraint improve generalization? Mechanically, it restricts the hypothesis space to curves a real SEI-limited cell can produce, so the model cannot buy training fit with a shape that fails after the split – capacity is derived from a film trajectory rather than drawn freely. Empirically the physics-informed model generalises far better than the plain tanh baseline: held-out RMSE falls from 0.0711 to 0.0457 A h (−36%) and MAPE from 4.32% to 2.78%. That figure should not be read as the physics term's contribution alone, however: as noted in Sections 1.3 and 2.3, the two models also differ by one hidden layer and a final activation, so the comparison is confounded and a matched-architecture ablation is still owed. The PINN does not beat LSTM/MLP (ReLU) on raw error (Table 1), but those two are unconstrained fitters whose lead (≈0.01 A h) is smaller than the physical violations they commit to earn it – MLP (ReLU) buys its 0.0104 A h edge with 9 spontaneous capacity increases.

5 Conclusion & Next Steps

5.1 Conclusion

Guiding the network with SEI-growth physics substantially improves its held-out fit over the same-loss-free network (Section 4.1), even though that gain is not yet cleanly isolated from an accompanying architecture change. That ceiling has an identified cause: the real cell has 21 cycles where capacity rises by up to 0.07 A h, driven by rest – lithium re-equilibrates during long pauses (`documents/step5_recovery_pinn.ipynb`) – and Section 2.2's SEI PINN is monotone by construction, so it structurally cannot fit those points; that mismatch is precisely why its held-out error cannot go below ∼2.78% MAPE no matter how the physics term is tuned.

On process, not just outcome. The governing equation here was not adopted by starting from the best-known published PINN and reproducing it. The priority was the reverse: understand what this specific dataset actually contains – one cell, one temperature, 131 cycles, and real rest-driven recovery events – and check whether the exact problem, on the exact data, had already been worked before designing around it (Section 1.2). Every modelling choice was kept pointed at the physical question being asked, not at whatever an LLM assistant might suggest as the most impressive result to extract from the dataset in front of us.

On raw held-out error the LSTM and MLP (ReLU) are effectively tied rather than either being the clear winner (Table 1), and neither succeeds by modelling the recovery mechanism: both are univariate curve-fitters over capacity alone, so the LSTM's apparent edge, where present, comes from extrapolating the local trend of that one series recursively, not from correlating capacity against any other measured variable.

5.2 Next steps

- **Matched-architecture ablation.** The PINN-vs-MLP(tanh) gap reported in Section 4.1 is confounded by an architecture change (one extra hidden layer, a `sigmoid` final activation) alongside the physics loss (Sections 1.3, 2.3). Training a PINN on the MLP's exact 3-layer, all-`tanh` shape would isolate what the physics term itself is worth.
- **Hard-monotone PINN (A2).** Section 4.1 shows the soft physics residual keeps capacity monotone over the evaluated range (0 violations, cycles 2–132) but not under heavy

extrapolation (32 violations by cycle 1000). Building A2 would remove that interior-monotonicity caveat by construction rather than by measurement.

- **A4 (EIS dual observable)** – attempt only if the parser yields clean R_e/R_{ct} (D-005, stretch goal).
- **Arrhenius temperature term.** E_a is currently fixed, not fitted, because a single ambient-temperature group cannot identify it (D-006). Cite a literature E_a value, or extend Eq. 1 once a multi-temperature or EIS-derived signal is available.

References

[1] F. Wang, Z. Zhai, Z. Zhao, et al. Physics-informed neural network for lithium-ion battery degradation stable modeling and prognosis. *Nature Communications* **15**, 4332 (2024). https://doi.org/10.1038/s41467-024-48779-z